

ESP32-S3 GDMA Bus-Contention Profiler

Design, measurements and findings

Parth Sinha

October 2026

Abstract

This report describes a profiler that measures how General DMA (GDMA) traffic on the ESP32-S3 interferes with 128-bit PIE SIMD workloads running on the other CPU core. Core 1 runs four vector kernels (read, write, copy and stride) over internal SRAM and external octal PSRAM, while Core 0 keeps the GDMA engine saturated with memory-to-memory transfers. A host script requests a 27-cell measurement matrix over a native USB vendor interface and produces a CSV file and a plot. When GDMA writes into PSRAM, every CPU workload in PSRAM slows down by 41–69 %, and the DMA throughput itself falls from 36.4 MB/s to 12–18 MB/s. SRAM workloads lose less than 8 % in every configuration. The same firmware streams GDMA-filled ping-pong buffers to the host at 1.18 MB/s, which is 97 % of the USB full-speed bulk limit, with no data loss under any Core 1 load.

Contents

1	Goals	2
2	Platform	2
2.1	Hardware	2
2.2	Software	2
2.3	Hardware limits that shaped the design	2
3	System architecture	2
4	Firmware design	3
4.1	GDMA load generator	3
4.2	PIE SIMD kernels	3
4.3	Measurement method	3
4.4	Ping-pong stream	4
4.5	USB configuration	4
5	Wire protocol	4
6	Host tools	4
7	Results	5
7.1	USB streaming	5
7.2	Contention matrix	5
7.3	Discussion	6
8	Problems found and fixed	6
9	Reproducing the results	7
10	Limitations and future work	7

1 Goals

The project set out to answer three questions:

1. How much does concurrent GDMA traffic slow a SIMD workload on the other core, and how does this depend on where the data lives (SRAM or PSRAM) and on the access pattern?
2. How much does the CPU workload, in turn, slow the DMA engine?
3. Can a GDMA-fed producer stream data to a PC over USB without loss while the other core is under heavy memory load?

The work was organised in five phases (0–4), summarised in Table 1.

Phase	Outcome
0 Baseline	GDMA M2M pump on Core 0, PIE read kernel on Core 1, first contention table
1 USB bulk	TinyUSB vendor class with bulk IN/OUT; per-packet ZLP problem fixed
2 Ping-pong	Two GDMA-filled 4 KB buffers streamed over USB; backpressure and paced modes
3 SIMD engine	Read, write, copy and stride kernels; full kernel $\times$ memory $\times$ GDMA matrix
4 Analytics	Profiling on request over USB; host computes rates, writes CSV and PNG

Table 1: Project phases.

2 Platform

2.1 Hardware

- ESP32-S3 N16R8 module: dual-core Xtensa LX7 at 240 MHz, 16 MB flash, 8 MB octal PSRAM at 80 MHz.
- 32 KB data cache with 32 B lines in front of PSRAM.
- Two USB connectors: a CH343 USB–UART bridge (`/dev/ttyACM1`) used for flashing and the console, and the native USB-OTG port, which enumerates as vendor device `303a:4020`.

2.2 Software

ESP-IDF v5.3, FreeRTOS with a 100 Hz tick, and the `espressif/esp_tinyusb` component (≥ 1.4). Host scripts are written in Python and run with `uv` using `pyusb` and `matplotlib`.

2.3 Hardware limits that shaped the design

Three constraints of the chip changed the original plan:

- **GDMA cannot feed USB-OTG.** The USB controller has its own DMA engine; GDMA serves SPI2/3, UHCI, I2S, LCD/CAM, AES, SHA, ADC, RMT and memory-to-memory transfers. The stream therefore uses GDMA to fill a buffer and the CPU to hand it to TinyUSB.
- **PIE SIMD is integer only.** There are no 128-bit floating-point vectors, so the kernels are pure 128-bit load/store streams.
- **USB is full speed only** (12 Mbit/s). The practical bulk ceiling is 1.216 MB/s.

3 System architecture

Work is split between the two cores so that the DMA load generator and the measured workload never share a CPU (Table 2).

Task	Core	Priority	Role
<code>dma_pump</code>	0	10	Keeps two GDMA M2M transfers queued while a DMA mode is active
<code>usb_stream</code>	0	6	Moves ready ping-pong buffers into TinyUSB
TinyUSB task	0	5	USB device stack (pinned to Core 0 in <code>sdkconfig.defaults</code>)
<code>profiler</code>	1	5	Runs the profile matrix, or a stream load kernel

Table 2: FreeRTOS tasks.

The host talks to the board over one bulk OUT and one bulk IN endpoint. It sends either a 4-byte PROF command, which starts a profile, or a 12-byte stream request (Section 5).

4 Firmware design

4.1 GDMA load generator

`dma_pump` uses the ESP-IDF async memcpy driver on a GDMA AHB channel. Each transfer moves 32 KB from an internal SRAM source buffer to either a second SRAM buffer or a PSRAM buffer. A counting semaphore with two slots keeps two transfers in flight, so the channel never idles between transfers; this raised SRAM-to-SRAM throughput from 59 to 61.6 MB/s. The completion ISR adds the transfer length to a byte counter, from which the host later derives the DMA rate.

4.2 PIE SIMD kernels

All four kernels are written in inline assembly with the Xtensa zero-overhead loop (`loopnez`) and the PIE post-increment instructions `ee.vld.128.ip` and `ee.vst.128.ip`. Each kernel sweeps the same address span, so cycles per KB are comparable.

Kernel	Access pattern
<code>k_read</code>	Four 16 B loads per loop iteration, contiguous
<code>k_write</code>	Four 16 B stores per loop iteration, contiguous
<code>k_copy</code>	Copies the first half of the buffer to the second half
<code>k_stride</code>	One 16 B load per 32 B cache line, so on PSRAM every load is a line miss

Table 3: PIE kernels.

Listing 1: Read kernel.

```
asm volatile(
    "loopnez %1, 1f\n"
    "ee.vld.128.ip q0, %0, 16\n"
    "ee.vld.128.ip q1, %0, 16\n"
    "ee.vld.128.ip q2, %0, 16\n"
    "ee.vld.128.ip q3, %0, 16\n"
    "1:\n"
    : "+r"(buf) : "r"(len / 64) : "memory");
```

Workload buffers are 64 KB in SRAM and 256 KB in PSRAM. The PSRAM buffer is eight times larger than the data cache, so every pass misses.

4.3 Measurement method

For each cell, `run_cell` sets the GDMA mode, waits 20 ms for the pump to reach steady state, then runs the kernel repeatedly. It runs at least 50 times and for at least 100 ms, so many DMA

completions fall inside the window. The CPU cycle counter gives the best and summed cycles per run; the DMA byte counter gives the bytes moved in the same window. The device sends only raw counters; the host derives every rate.

The matrix has 27 cells: four kernels on SRAM, the read kernel on the GDMA source buffer (SRAM*), and four kernels on PSRAM, each under three GDMA modes (idle, SRAM→SRAM, SRAM→PSRAM).

4.4 Ping-pong stream

Two 4KB DMA-capable SRAM buffers move through the states FREE → FILLING → READY. `produce()` claims the next free buffer, writes an 8-byte header {`seq`, `overruns`}, and starts a GDMA copy of a fixed incrementing-word pattern into the rest of the block. The completion ISR marks the buffer ready and notifies `usb_stream`, which passes it to `tud_vendor_write` and frees it.

- **Backpressure mode** (interval 0): the next block is produced as soon as a buffer is free. Nothing is lost and throughput is limited by USB.
- **Paced mode** (interval > 0): an `esp_timer` produces one block per tick, as a sampling peripheral would. A tick that finds its buffer still owned by USB counts an overrun and advances `seq`, so the host sees exactly one sequence gap per dropped block.

4.5 USB configuration

TinyUSB’s buffered vendor mode drains a 64B FIFO and appends a zero-length packet after every packet. The top-level `CMakeLists.txt` switches to unbuffered mode with 4KB IN transfers (`CFG_TUD_VENDOR_TXRX_BUFFERED=0`, `CFG_TUD_VENDOR_TX_EPSIZE=4096`), so each block is one transfer.

5 Wire protocol

Direction	Message	Layout (little endian)
OUT	Stream request	u32 <code>blocks</code> , u32 <code>interval_us</code> , u32 <code>load</code> ; <code>load</code> = <code>kernel</code> 0x10 for PSRAM
IN	Stream block	u32 <code>seq</code> , u32 <code>overruns</code> , then 4088 B pattern
OUT	Profile request	ASCII PROF (4 bytes)
IN	Profile reply	One 4096 B block: ‘PROF’, u32 <code>count</code> , u32 <code>cpu_mhz</code> , u32 32, then <code>count</code> cells

Table 4: USB messages.

Listing 2: Profile cell (32 bytes).

```
typedef struct {
    uint8_t mem, kernel, gdma, pad;
    uint32_t len, runs, best, elapsed, dma_bytes; // cycles, except len/dma_bytes
    uint64_t sum;                               // cycles over all runs
} cell_t;
```

6 Host tools

- `host/usbdev.py` opens 303a:4020 and drains data left over from an interrupted run.
- `host/stream.py` [`blocks`] [`interval_us`] [`load`] measures throughput, checks every block against the pattern, and verifies that sequence gaps equal the device’s overrun count.

- `host/profile.py [prefix] [--replot]` requests the matrix, prints it with slowdown relative to the idle bus, and writes `<prefix>.csv` and `<prefix>.png`.

7 Results

7.1 USB streaming

Mode	Offered (MB/s)	Delivered (MB/s)	Dropped
Backpressure	–	1.182	0
Paced, 5000 μ s	0.819	0.818	0
Paced, 4000 μ s	1.024	1.022	0
Paced, 3000 μ s	1.365	1.180	76 of 576 (13.2 %)

Table 5: Stream throughput. In the overloaded case, sequence gaps matched the device overrun count exactly.

Running any of the eight kernel/memory loads on Core 1 during the stream changed neither the stream throughput (1.18 MB/s in backpressure mode) nor the drop count (zero at 3500 μ s), and the Core 1 rates were unchanged as well. The stream uses about 2% of SRAM bandwidth, and USB full speed is the bottleneck, so contention cannot appear on this path.

7.2 Contention matrix

Table 6 gives average cycles per KB for each workload, with the change relative to an idle bus. Figure 1 shows the same data.

Memory	Kernel	CPU idle	idle	GDMA SRAM→SRAM		GDMA SRAM→PSRAM	
		MB/s	c/KB	c/KB	%	c/KB	%
SRAM	read	3821.1	64.3	66.6	+3.6	64.4	+0.1
SRAM	write	3821.4	64.3	66.6	+3.6	64.4	+0.1
SRAM	copy	3818.3	64.4	68.9	+7.0	64.7	+0.5
SRAM	stride	7609.1	32.3	33.5	+3.6	32.3	+0.1
SRAM*	read	3804.8	64.6	69.5	+7.6	67.1	+3.9
PSRAM	read	62.4	3939.7	3939.6	0.0	6659.5	+69.0
PSRAM	write	35.1	7004.9	7012.5	+0.1	9853.3	+40.7
PSRAM	copy	44.5	5527.5	5523.6	−0.1	8260.3	+49.4
PSRAM	stride	62.3	3943.1	3939.3	−0.1	6657.9	+68.8

Table 6: Average CPU cycles per KB at 240 MHz. SRAM* means the CPU reads the GDMA source buffer. CPU MB/s is address span per second, so stride, which touches half the bytes, shows twice the read rate.

CPU workload	GDMA SRAM→SRAM (MB/s)	GDMA SRAM→PSRAM (MB/s)
Any SRAM workload	61.6	36.4
PSRAM read	61.8	18.1
PSRAM write	61.7	12.3
PSRAM copy	61.6	14.7
PSRAM stride	61.6	18.0

Table 7: GDMA throughput while the CPU runs each workload.

ESP32-S3 bus contention: PIE SIMD (Core 1) vs GDMA (Core 0) @ 240 MHz

Worst case: PSRAM read with GDMA sram->psram is +69% slower. SRAM workloads stay under 8%. SRAM* = CPU reads the GDMA source buffer.

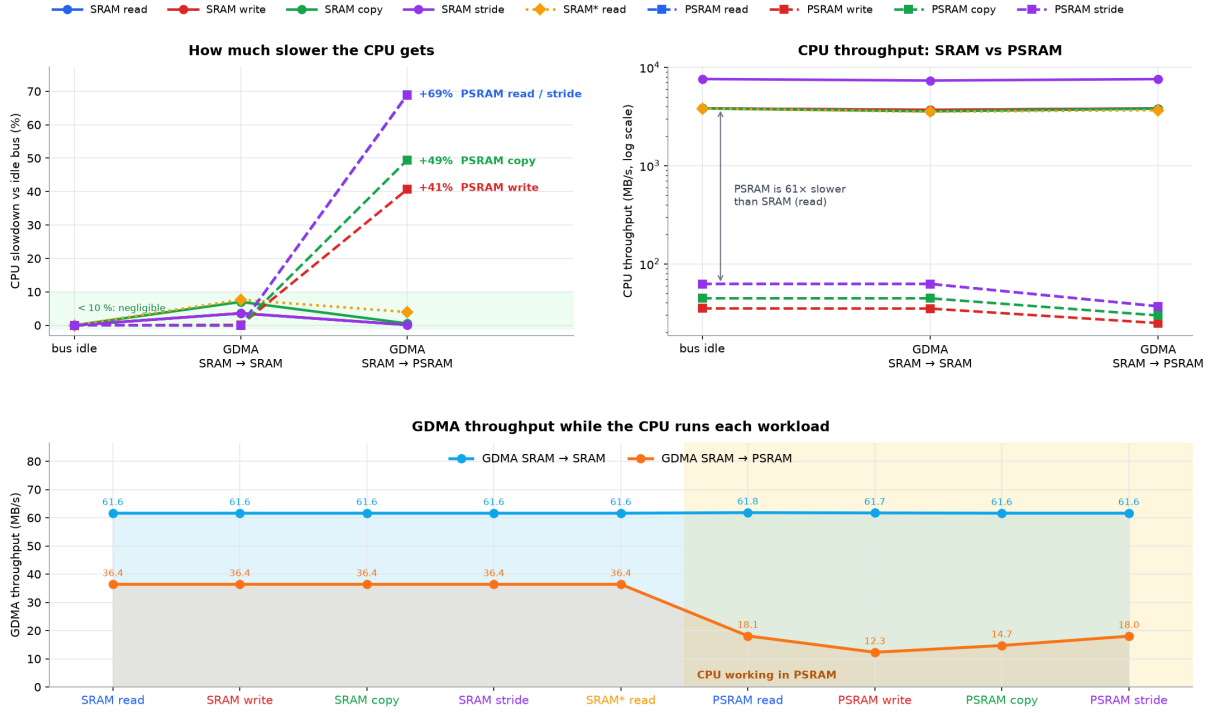


Figure 1: Contention profile. Top left: CPU slowdown relative to an idle bus. Top right: CPU throughput on a log scale. Bottom: GDMA throughput while the CPU runs each workload.

7.3 Discussion

- **SRAM costs one cycle per 16 bytes** regardless of access pattern. GDMA traffic into the same SRAM adds 2–7 %, and up to 7.6 % when the CPU reads the very buffer GDMA is reading. GDMA into PSRAM barely touches SRAM workloads.
- **PSRAM is bound by cache-line fills.** The stride kernel, which loads only one 16 B vector per line, costs the same as reading every byte. Writes cost about 1.8× reads because each line is both filled and later written back. With an idle bus, reading SRAM is about 61× faster than reading PSRAM.
- **GDMA into PSRAM is the dominant source of contention.** CPU workloads in PSRAM slow by 41–69 %, and the effect is mutual: GDMA throughput into PSRAM falls from 36.4 MB/s to 12–18 MB/s, with the write kernel, which generates the most PSRAM traffic, hurting it most.
- **GDMA SRAM→SRAM does not affect PSRAM workloads,** and PSRAM workloads do not slow it.

The practical conclusion is that data placement matters far more than DMA scheduling. Keeping either the CPU’s working set or the DMA destination in internal SRAM removes nearly all of the interference.

8 Problems found and fixed

Zero-length packet after every USB packet.

TinyUSB’s buffered vendor mode sent a ZLP after each 64 B packet. Switching to unbuffered mode with 4 KB transfers fixed it.

10 ms per block in backpressure mode (0.41 MB/s).

The streaming task busy-pollled at idle priority, so the idle task on Core 0 executed `waiti` and slept until the next 10ms tick. The task now blocks on notifications from the GDMA ISR and from `tud_vendor_tx_cb`, and runs at priority 6. Throughput rose to 1.18 MB/s.

Stream fell to 0.41 MB/s under any Core 1 load.

The TinyUSB task defaulted to Core 1 at priority 5 and shared time slices with the busy workload. Pinning it to Core 0 (`CONFIG_TINYUSB_TASK_AFFINITY_CPU0`) fixed it.

`esp_timer_stop` inside a critical section.

Moved outside the spinlock.

9 Reproducing the results

One-time setup: add the user to dialout and install a udev rule for 303a:4020 (see `README.md`). Then, from the project folder:

```
. ~/esp/esp-idf/export.sh
idf.py -p /dev/ttyACM1 flash monitor          # Ctrl+] exits

# in a second terminal
uv run --with pyusb host/stream.py 1000        # max throughput
uv run --with pyusb host/stream.py 500 3000    # paced, overruns
uv run --with pyusb host/stream.py 500 3500 psram:write # stream under load
uv run --with pyusb --with matplotlib host/profile.py results/profile
```

Each stream test prints `data OK` when it passes. To rebuild this report, run `pdflatex report.tex` twice in `docs/`.

10 Limitations and future work

- The stream producer is a memory-to-memory copy driven by a timer. A real peripheral source (I2S RX or continuous ADC) would exercise a true peripheral DMA path.
- Placing the ping-pong buffers in PSRAM would make the stream compete with PSRAM workloads.
- GDMA burst size, transfer size and PSRAM clock (80 versus 120 MHz) were not swept.
- The matrix has no “USB active” dimension, because the stream showed no measurable effect.